Thesis: A Comparative Study of Large Language Models for Financial Sentiment Analysis and Their Predictive Potential for Short-Term Stock Price Movement

Component: Experiment 2 - Model D: Gemma 3 (4B Instruct)

Description: This notebook loads the prepared 1000 headline dataset and evaluates Gemma 3 4B on Experiment 2 using zero-shot prompting. It maps sentiment predictions to directional signals and computes directional accuracy, precision, recall, and F1-score based on next-day stock price movement.

Select T4 GPU as runtime.

```
# Install bitsandbytes — restart required after this

!pip install -q -U bitsandbytes accelerate
```

Go to runtime -> restart this session again -> then run cell 1 and run cell 2

```
# Import required libraries — Experiment 2d: Gemma 3

import pandas as pd
import numpy as np
import torch
import warnings
import gc

from transformers import AutoTokenizer, AutoModelForCausalLM, BitsAndBytesConfig
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score,
    f1_score, classification_report
)

warnings.filterwarnings("ignore")

print(f"PyTorch version : {torch.__version__}")
print(f"CUDA available  : {torch.cuda.is_available()}")
if torch.cuda.is_available():
    print(f"GPU detected    : {torch.cuda.get_device_name(0)}")
```

```
PyTorch version : 2.10.0+cu128
CUDA available  : True
GPU detected    : Tesla T4
```

Add the API Key from Hugging Face using "Add New Secret" in Google Colab

```
# Connect to Hugging Face and load Experiment 2 dataset

from google.colab import userdata, drive
from huggingface_hub import login

drive.mount("/content/drive")

hf_token = userdata.get("HF_TOKEN")
login(token=hf_token)

df_exp2 = pd.read_csv("/content/drive/MyDrive/Thesis_Data/exp2_dataset.csv")

print("Dataset loaded.")
print(f"Total headlines : {len(df_exp2)}")
print(f"\nMovement distribution:")
print(df_exp2["movement"].value_counts())
```

```
Mounted at /content/drive
Dataset loaded.
Total headlines : 1000

Movement distribution:
movement
1    508
0    492
Name: count, dtype: int64
```

```
# Load Gemma 3 4B

bnb_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_quant_type="nf4",
```

```
    bnb_4bit_compute_dtype=torch.float16,
    bnb_4bit_use_double_quant=True
)

print("Loading Gemma 3 4B...")

gemma_tokenizer = AutoTokenizer.from_pretrained(
    "google/gemma-3-4b-it",
    token=hf_token
)

gemma_model = AutoModelForCausalLM.from_pretrained(
    "google/gemma-3-4b-it",
    quantization_config=bnb_config,
    device_map="auto",
    token=hf_token
)

print("Gemma 3 4B loaded successfully.")
```

```
Loading Gemma 3 4B...
config.json: 100%                                                 855/855 [00:00<00:00, 15.7kB/s]
tokenizer_config.json:        1.16M/? [00:00<00:00, 13.2MB/s]
tokenizer.json: 100%                                              33.4M/33.4M [00:01<00:00, 168MB/s]
added_tokens.json: 100%                                             35.0/35.0 [00:00<00:00, 761B/s]
special_tokens_map.json: 100%                                       662/662 [00:00<00:00, 19.6kB/s]
model.safetensors.index.json:         90.6k/? [00:00<00:00, 1.90MB/s]
Download complete: 100%         8.60G/8.60G [04:13<00:00, 40.6MB/s]
Fetching 2 files: 100%                                              2/2 [04:02<00:00, 242.36s/it]
Loading weights: 100%                 883/883 [00:36<00:00, 560.89it/s, Materializing param=model.vision_tower.vision_model.post_layernorm.weight]
generation_config.json: 100%                                        215/215 [00:00<00:00, 14.7kB/s]
Gemma 3 4B loaded successfully.
```

```
# Define Gemma 3 classifier — positive or negative only

import logging
logging.getLogger("transformers").setLevel(logging.ERROR)

def classify_gemma(text):
    messages = [
        {"role": "user", "content": f"You are a financial sentiment classifier. Classify the sentiment of this f
    ]

    inputs = gemma_tokenizer.apply_chat_template(
        messages,
        add_generation_prompt=True,
        return_tensors="pt",
        return_dict=True
    ).to(gemma_model.device)

    with torch.no_grad():
        outputs = gemma_model.generate(
            **inputs,
            max_new_tokens=5,
            do_sample=False
        )

    input_length = inputs["input_ids"].shape[1]
    generated = gemma_tokenizer.decode(
        outputs[0][input_length:],
        skip_special_tokens=True
    ).strip().lower()

    if "negative" in generated:
        return "negative"
    else:
        return "positive"

print("Gemma 3 classifier ready.")
```

```
Gemma 3 classifier ready.
```

```
# Run Gemma 3 on 1,000 headlines dataset
```

```
headlines = df_exp2["headline"].tolist()
gemma_preds = []
total = len(headlines)

print(f"Running Gemma 3 on {total} headlines...")
print("-" * 40)

for i, text in enumerate(headlines):
    label = classify_gemma(text)
    gemma_preds.append(label)

    if (i + 1) % 100 == 0:
        print(f"  Progress: {i+1}/{total}")

print(f"\nDone. Total predictions: {len(gemma_preds)}")
print(f"\nSentiment distribution:")
print(pd.Series(gemma_preds).value_counts())
```

```
Running Gemma 3 on 1000 headlines...
----------------------------------------
  Progress: 100/1000
  Progress: 200/1000
  Progress: 300/1000
  Progress: 400/1000
  Progress: 500/1000
  Progress: 600/1000
  Progress: 700/1000
  Progress: 800/1000
  Progress: 900/1000
  Progress: 1000/1000

Done. Total predictions: 1000

Sentiment distribution:
positive    526
negative    474
Name: count, dtype: int64
```

```
# Map sentiment to directional prediction and evaluate

df_exp2["gemma_sentiment"] = gemma_preds
df_exp2["gemma_direction"] = df_exp2["gemma_sentiment"].map({
    "positive": 1,
    "negative": 0
})

true_movement = df_exp2["movement"].tolist()
pred_movement = df_exp2["gemma_direction"].tolist()

acc  = accuracy_score(true_movement, pred_movement)
prec = precision_score(true_movement, pred_movement, average="macro")
rec  = recall_score(true_movement, pred_movement, average="macro")
f1   = f1_score(true_movement, pred_movement, average="macro")

print("=" * 50)
print("Gemma 3 4B — Experiment 2 Results (next-day movement)")
print("=" * 50)
print(f"  Directional Accuracy : {acc:.4f}  ({acc*100:.2f}%)")
print(f"  Precision            : {prec:.4f}")
print(f"  Recall               : {rec:.4f}")
print(f"  F1-Score             : {f1:.4f}")
print("=" * 50)
print(classification_report(true_movement, pred_movement,
      target_names=["Down (0)", "Up (1)"]))
```

```
==================================================
Gemma 3 4B — Experiment 2 Results (next-day movement)
==================================================
  Directional Accuracy : 0.4900  (49.00%)
  Precision            : 0.4896
  Recall               : 0.4896
  F1-Score             : 0.4894
==================================================
              precision    recall  f1-score   support

    Down (0)       0.48      0.46      0.47       492
      Up (1)       0.50      0.52      0.51       508

    accuracy                           0.49      1000
   macro avg       0.49      0.49      0.49      1000
weighted avg       0.49      0.49      0.49      1000
```

```
# Save Gemma 3 Experiment 2 results

gemma_exp2_scores = {
    "model": "Gemma 3 4B",
    "directional_accuracy": round(acc, 4),
    "precision": round(prec, 4),
    "recall": round(rec, 4),
    "f1_score": round(f1, 4)
}

print("Gemma 3 4B — Experiment 2 Results Summary")
print(pd.DataFrame([gemma_exp2_scores]))

df_exp2.to_csv("/content/drive/MyDrive/Thesis_Data/exp2_gemma_preds.csv", index=False)
print("\nSaved to Google Drive.")
```

```
Gemma 3 4B — Experiment 2 Results Summary
        model  directional_accuracy  precision  recall  f1_score
0  Gemma 3 4B                  0.49     0.4896  0.4896    0.4894

Saved to Google Drive.
```